Lab Report Task 2 - 5

1. Task 2: Encrypting a Message

โดยเป้าหมายหลักของ Task นี้คือ ทำความเข้าใจกระบวนการซ่อนความลับโดยใช้กุญแจสาธารณะ โดยใน Task นี้ เราจำลองสถานการณ์ว่าเราต้องการส่งข้อความลับ "A top secret!" ไปหาใครสักคน กฎของ RSA คือถ้าอยากส่งข้อความลับให้ใครจะต้องใช้ Public Key ของคนนั้นในการเข้ารหัส โดยเราทำการแปลงข้อความเป็นตัวเลขฐาน 16 แล้วนำไปเข้าสมการคณิตศาสตร์ $C \equiv M^e \pmod{n}$ สิ่งสำคัญคือ เมื่อข้อความถูกเข้ารหัสกลายเป็น Ciphertext $\boldsymbol{C}$ แล้ว มันจะกลายเป็นชุดตัวเลขที่อ่านไม่ออก และ Public Key ที่ใช้เข้ารหัสไปเมื่อครู่จะไม่สามารถนำมาถอดรหัสข้อความนี้ได้อีก

a. คำสั่งการแปลงข้อความใน Python

คำสั่งนี้ทำหน้าที่ในการแปลงข้อความชนิด ASCII ให้อยู่ในรูปแบบเลขฐาน 16

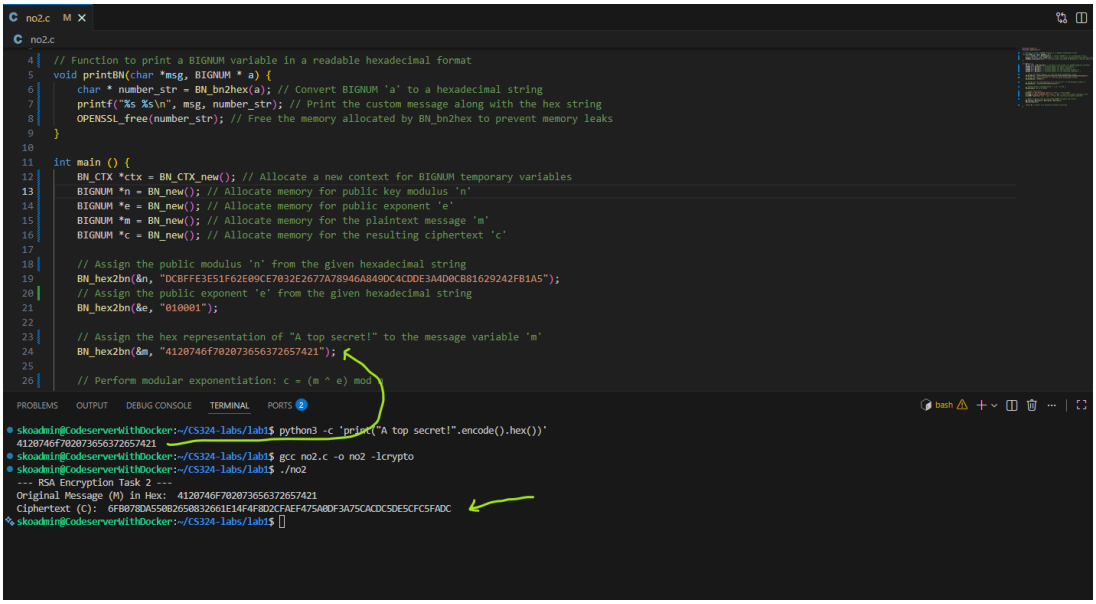
```
# python -c: executes the command provided in the string
# "A top secret!".encode("hex"): converts the ASCII string to its
hexadecimal representation
# Note: This command is for python3 only (linux based)
$ python3 -c 'print("A top secret!".encode().hex())'
```

b. โค้ดการทำงานในภาษา c (มีในไฟล์แยก)

```
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM variable in a readable hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a); // Convert BIGNUM 'a' to a
hexadecimal string
    printf("%s %s\n", msg, number_str); // Print the custom message along
with the hex string
    OPENSSL_free(number_str); // Free the memory allocated by BN_bn2hex
to prevent memory leaks
}

int main () {
    BN_CTX *ctx = BN_CTX_new(); // Allocate a new context for BIGNUM
temporary variables
    BIGNUM *n = BN_new(); // Allocate memory for public key modulus 'n'
    BIGNUM *e = BN_new(); // Allocate memory for public exponent 'e'
    BIGNUM *m = BN_new(); // Allocate memory for the plaintext message
'm'
    BIGNUM *c = BN_new(); // Allocate memory for the resulting ciphertext
'c'

    // Assign the public modulus 'n' from the given hexadecimal string
```

```
    BN_hex2bn(&n,
"DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    // Assign the public exponent 'e' from the given hexadecimal string
    BN_hex2bn(&e, "010001");

    // Assign the hex representation of "A top secret!" to the message
variable 'm'
    BN_hex2bn(&m, "4120746f702073656372657421");

    // Perform modular exponentiation: c = (m ^ e) mod n
    BN_mod_exp(c, m, e, n, ctx);

    // Output the results
    printf("--- RSA Encryption Task 2 ---\n"); // Print header
    printBN("Original Message (M) in Hex: ", m); // Print the original
message in hex
    printBN("Ciphertext (C): ", c); // Print the resulting encrypted
ciphertext

    // Memory cleanup: free all allocated BIGNUM variables and context
    BN_free(n); BN_free(e); BN_free(m); BN_free(c);
    BN_CTX_free(ctx);

    return 0; // Return 0 to indicate successful execution
}
```

c. ภาพแสดงการทำงาน



d. การอภิปราย

จากการทดลอง เมื่อเราแปลงข้อความ "A top secret!" เป็นเลขฐานสิบหกได้แล้ว จะได้ค่าตัวเลขขนาดใหญ่ (BIGNUM) เพื่อนำเข้าสู่สมการเข้ารหัสด้วย Public Key (e, n) ผ่านฟังก์ชัน `BN_mod_exp` ผลลัพธ์ที่ได้คือ Ciphertext ซึ่งเป็นชุดตัวเลขฐานสิบหกที่ไม่สามารถอ่านเป็นข้อความ

เดิมได้โดยตรง กระบวนการนี้รับประกัน Confidentiality (ความลับของข้อมูล) เนื่องจากผู้ที่จะสามารถย้อนกระบวนการเพื่อหาค่า M กลับมาได้ จะต้องมี Private Key d เท่านั้น

2. Task 3: Decrypting a Message

เป้าหมายหลักของ Task นี้คือเพื่อพิสูจน์ว่ามีเพียงเจ้าของกุญแจส่วนตัวเท่านั้นที่สามารถอ่านข้อความลับได้ ต่อเนื่องจาก Task 2 เมื่อ Ciphertext C ถูกส่งมาถึงผู้รับ ผู้รับจะใช้ Private Key d ของตนเองซึ่งเป็นความลับสุดยอดและไม่มีใครรู้ มาทำการถอดรหัสผ่านโดยใช้สมการ $M \equiv C^d \pmod{n}$ Task นี้แสดงให้เห็นถึงความสัมพันธ์ทางคณิตศาสตร์ที่ผูกมัดระหว่าง Public Key และ Private Key อย่างแยกไม่ออก หากมีแฮกเกอร์ขโมย Ciphertext ไปได้ระหว่างทาง พวกเขาก็อ่านไม่ออกอยู่ดีเพราะไม่มีค่า d

a. โค้ดการทำงานในภาษา c (มีในไฟล์แยก)

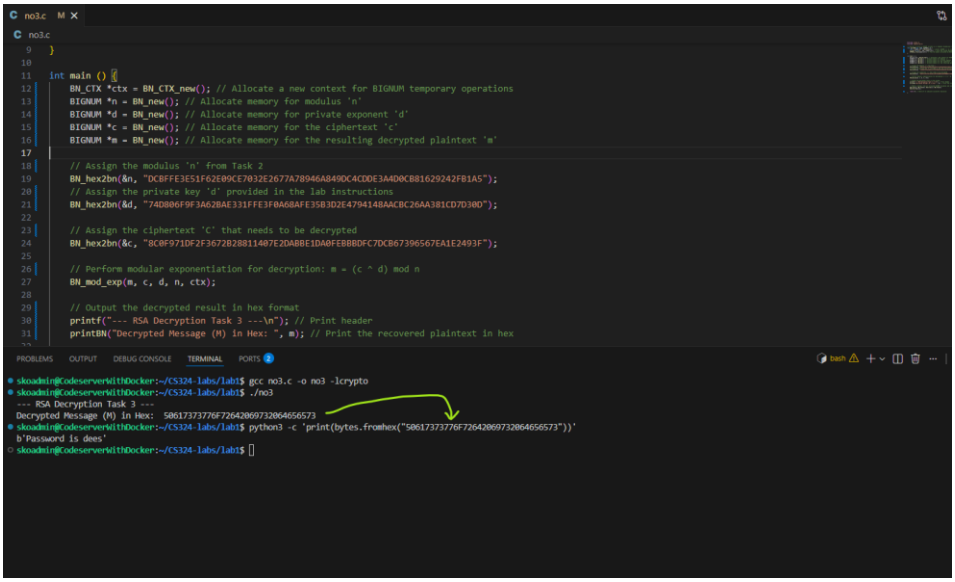
```
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM variable in a readable hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a); // Convert BIGNUM 'a' to a
hexadecimal string
    printf("%s %s\n", msg, number_str); // Print the custom message along
with the hex string
    OPENSSL_free(number_str); // Free the memory allocated by BN_bn2hex
}

int main () {
    BN_CTX *ctx = BN_CTX_new(); // Allocate a new context for BIGNUM
temporary operations
    BIGNUM *n = BN_new(); // Allocate memory for modulus 'n'
    BIGNUM *d = BN_new(); // Allocate memory for private exponent 'd'
    BIGNUM *c = BN_new(); // Allocate memory for the ciphertext 'c'
    BIGNUM *m = BN_new(); // Allocate memory for the resulting decrypted
plaintext 'm'

    // Assign the modulus 'n' from Task 2
    BN_hex2bn(&n,
"DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    // Assign the private key 'd' provided in the lab instructions
    BN_hex2bn(&d,
"74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

    // Assign the ciphertext 'C' that needs to be decrypted
    BN_hex2bn(&c,
"8C0F971DF2F3672B28811407E2DABBE1DA0FEBBBDFC7DCB67396567EA1E2493F");
```

```
    // Perform modular exponentiation for decryption: m = (c ^ d) mod n
    BN_mod_exp(m, c, d, n, ctx);

    // Output the decrypted result in hex format
    printf("--- RSA Decryption Task 3 ---\n"); // Print header
    printBN("Decrypted Message (M) in Hex: ", m); // Print the recovered
plaintext in hex

    // Memory cleanup: free all allocated BIGNUM variables and context
    BN_free(n); BN_free(d); BN_free(c); BN_free(m);
    BN_CTX_free(ctx);

    return 0; // Return 0 to indicate successful execution
}
```

b. คำสั่งการแปลงรหัสใน Python

```
# python -c: executes the command provided in the string
# "HEX_STRING".decode("hex"): decodes the hex string back to the original
ASCII text
$ python3 -c 'print(bytes.fromhex("50617373776F72642069732064656573"))'
```

c. ผลการรัน



d. อภิปรายผล

การถอดรหัสใน Task 3 แสดงให้เห็นว่า เมื่อนำ Ciphertext ที่ถูกสร้างจากการทำงานของโมดูโล ไปยกกำลังด้วย Private Key d ของผู้รับ จะสามารถดึงข้อมูล Plaintext ต้นฉบับกลับมาได้อย่างสมบูรณ์แบบ ซึ่งสามารถหลักฐานยืนยันความสัมพันธ์ทางคณิตศาสตร์ของ RSA ซึ่งกุญแจทั้งสอง (Public/Private) ทำงานร่วมกันอย่างเป็นระบบ หาก d ผิดพลาดไปเพียงนิดเดียว ค่าที่ได้จาก $C^d \pmod{n}$ จะออกมาเป็นขยะทันที

3. Task 4: Signing a Message

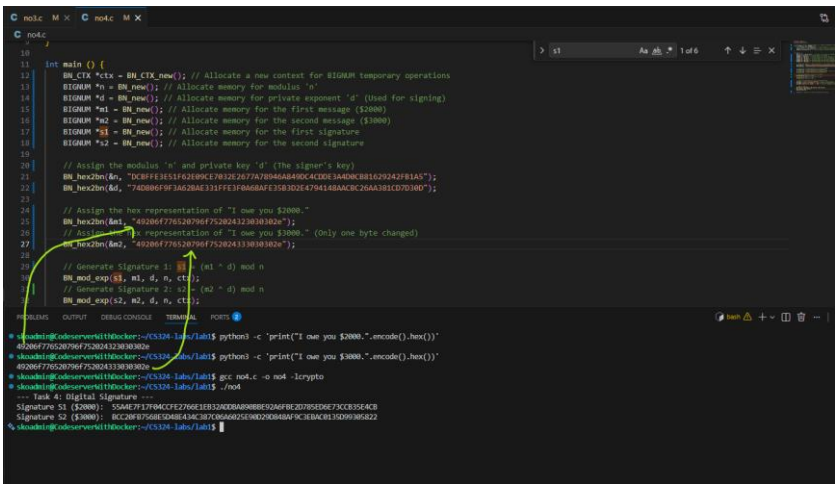
ใน Task นี้ เป้าหมายหลักคือเพื่อทำความเข้าใจกลไกการรับรองความถูกต้องของข้อมูล และการป้องกันการปฏิเสธความรับผิดชอบ โดยการทำ Digital Signature จะทำงาน กลับกันกับการเข้ารหัสใน Task 2/3 ในกรณีนี้ เราไม่ได้ต้องการซ่อนข้อความ (ทุกคนอ่านข้อความได้) แต่เราต้องการพิสูจน์ว่าข้อความนี้มาจากเราจริง ๆ และไม่ได้ถูกแก้ไข เราจะใช้ Private Key d ของเราเอง นำมาเข้ากระบวนการทางคณิตศาสตร์กับข้อความ เพื่อสร้างเป็นลายเซ็น S มีเหตุผลเล็กน้อยว่าทำไมโจทย์ถึงให้เปลี่ยนข้อความจาก $2000 เป็น $3000 เพื่อให้เห็นว่า Digital Signature ผูกติดกับเนื้อหาของข้อความแบบ 100% หากข้อความเปลี่ยนไปแค่บิตเดียว เช่น โดนแฮกเกอร์แอบแก้ตัวเลขเงินกลางทาง ลายเซ็นที่คำนวณได้จะเปลี่ยนไปเป็นคนละเรื่องทันที ไม่สามารถเอาลายเซ็นเดิมมาแปะสวมรอยได้

a. โค้ดการทำงานในภาษา c (มีในไฟล์แยก)

```
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM variable in a readable hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a); // Convert BIGNUM 'a' to a
hexadecimal string
    printf("%s %s\n", msg, number_str); // Print the custom message along
with the hex string
    OPENSSL_free(number_str); // Free the memory allocated by BN_bn2hex
}

int main () {
    BN_CTX *ctx = BN_CTX_new(); // Allocate a new context for BIGNUM
temporary operations
    BIGNUM *n = BN_new(); // Allocate memory for modulus 'n'
    BIGNUM *d = BN_new(); // Allocate memory for private exponent 'd'
(Used for signing)
    BIGNUM *m1 = BN_new(); // Allocate memory for the first message
($2000)
    BIGNUM *m2 = BN_new(); // Allocate memory for the second message
($3000)
    BIGNUM *s1 = BN_new(); // Allocate memory for the first signature
    BIGNUM *s2 = BN_new(); // Allocate memory for the second signature

    // Assign the modulus 'n' and private key 'd' (The signer's key)
    BN_hex2bn(&n,
"DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&d,
"74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

    // Assign the hex representation of "I owe you $2000."
    BN_hex2bn(&m1, "49206f776520796f752024323030302e");
```

```
    // Assign the hex representation of "I owe you $3000." (Only one byte
changed)
    BN_hex2bn(&m2, "49206f776520796f752024333030302e");

    // Generate Signature 1: s1 = (m1 ^ d) mod n
    BN_mod_exp(s1, m1, d, n, ctx);
    // Generate Signature 2: s2 = (m2 ^ d) mod n
    BN_mod_exp(s2, m2, d, n, ctx);

    // Output both signatures to observe the differences
    printf("--- Task 4: Digital Signature ---\n"); // Print header
    printBN("Signature S1 ($2000): ", s1); // Print the signature for the
original message
    printBN("Signature S2 ($3000): ", s2); // Print the signature for the
modified message

    // Memory cleanup
    BN_free(n); BN_free(d); BN_free(m1); BN_free(m2);
    BN_free(s1); BN_free(s2);
    BN_CTX_free(ctx);

    return 0; // Return 0 to indicate successful execution
}
```

b. **ภาพหน้าจอ**



c. **การอภิปราย**

สิ่งที่สังเกตได้ โดยเมื่อเราปรับเปลี่ยนข้อความเพียงเล็กน้อยจาก "$2000" เป็น "$3000" (เปลี่ยนแปลงแค่ไม่กี่บิตในระดับฐานข้อมูล) ค่า Digital Signature (S1 และ S2) ที่คำนวณออกมามีความแตกต่างกันอย่างสิ้นเชิงในทุก ๆ บิต ลักษณะนี้แสดงให้เห็นถึงคุณสมบัติของการดำเนินการทางคณิตศาสตร์แบบ Modular Exponentiation ใน RSA เมื่อนำ Message มาเข้ารหัสด้วย Private Key d ผลลัพธ์จะกระจายตัวอย่างมาก หากมีการปลอมแปลงข้อความแม้แต่นิดเดียว ลายเซ็นใหม่ที่

ได้จะไม่ตรงกับลายเซ็นเดิมเลย คุณสมบัตินี้ช่วยปกป้อง Integrity (ความสมบูรณ์ของข้อมูล) และป้องกันไม่ให้ผู้ไม่หวังดี (Attacker) คัดลอกลายเซ็นเดิมไปแปะใส่ข้อความที่ถูกดัดแปลงได้

4. Task 5: Verifying a Signature

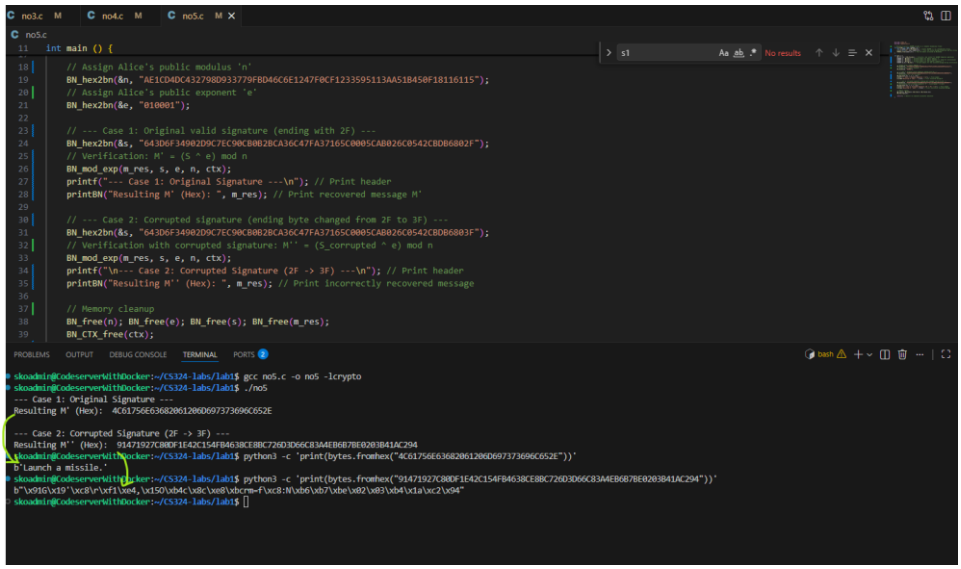
ใน Task นี้เป้าหมายหลักคือ เพื่อทำความเข้าใจกระบวนการยืนยันตัวตนผู้ส่งและทดสอบระบบเมื่อมีการโจมตีหรือข้อมูลพัง โดยเมื่อผู้รับ (Bob) ได้รับข้อความพร้อมลายเซ็นจากผู้ส่ง (Alice) Bob จะใช้ Public Key ของ Alice ในการตรวจสอบผ่านสมการ $M' \equiv S^e \pmod{n}$ ถ้าผลลัพธ์ที่ถอดออกมา M' ตรงกับข้อความต้นฉบับ แปลว่าลายเซ็นนี้มาจาก Alice แน่นอน เพราะมีแค่ Alice ที่มี Private Key ที่สร้างลายเซ็นนี้ได้ เหตุผลที่โจทย์ให้แก้ลายเซ็นแค่ 1 Bit จาก 2F เป็น 3F เนื่องด้วยเพื่อจำลองสถานการณ์ว่าหากลายเซ็นถูกปลอมแปลง หรือไฟล์เกิดเสียระหว่างทางเพียงแค่นิดเดียว กระบวนการตรวจสอบจะล้มเหลวทันที (ผลลัพธ์ที่ได้จะเป็นขยะ ไม่ตรงกับข้อความเดิม) ซึ่งเป็นกลไกที่แจ้งเตือนผู้รับว่าอย่าเชื่อถือข้อมูลชุดนี้

A. โค้ดการทำงานในภาษา c (มีในไฟล์แยก)

```
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM variable in a readable hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a); // Convert BIGNUM 'a' to a
hexadecimal string
    printf("%s %s\n", msg, number_str); // Print the custom message along
with the hex string
    OPENSSL_free(number_str); // Free the memory allocated by BN_bn2hex
}

int main () {
    BN_CTX *ctx = BN_CTX_new(); // Allocate a new context for BIGNUM
temporary operations
    BIGNUM *n = BN_new(); // Allocate memory for public modulus 'n'
    BIGNUM *e = BN_new(); // Allocate memory for public exponent 'e'
(Used for verification)
    BIGNUM *s = BN_new(); // Allocate memory for the digital signature
's'
    BIGNUM *m_res = BN_new(); // Allocate memory for the recovered
message during verification

    // Assign Alice's public modulus 'n'
    BN_hex2bn(&n,
"AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
    // Assign Alice's public exponent 'e'
    BN_hex2bn(&e, "010001");

    // --- Case 1: Original valid signature (ending with 2F) ---
```

```
    BN_hex2bn(&s,
"643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F");
    // Verification: M' = (S ^ e) mod n
    BN_mod_exp(m_res, s, e, n, ctx);
    printf("--- Case 1: Original Signature ---\n"); // Print header
    printBN("Resulting M' (Hex): ", m_res); // Print recovered message M'

    // --- Case 2: Corrupted signature (ending byte changed from 2F to
3F) ---
    BN_hex2bn(&s,
"643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
    // Verification with corrupted signature: M'' = (S_corrupted ^ e) mod
n
    BN_mod_exp(m_res, s, e, n, ctx);
    printf("\n--- Case 2: Corrupted Signature (2F -> 3F) ---\n"); //
Print header
    printBN("Resulting M'' (Hex): ", m_res); // Print incorrectly
recovered message

    // Memory cleanup
    BN_free(n); BN_free(e); BN_free(s); BN_free(m_res);
    BN_CTX_free(ctx);

    return 0; // Return 0 to indicate successful execution
}
```

B. ภาพถ่ายหน้าจอ



C. การอภิปราย

ผลลัพธ์ของกระบวนการตรวจสอบเมื่อมีการเปลี่ยนแปลงข้อมูล 1 Bit ลายเซ็นต้นฉบับด้วย Public Key ของ Alice เราจะได้ข้อความต้นฉบับกลับมา (ตรวจสอบแล้วตรงกับคำว่า "Launch a missle.") ซึ่งแสดงถึง Authentication ว่าข้อความนี้มาจาก Alice จริงๆ และ Non-repudiation (การปฏิเสธความรับผิดชอบ

ไม่ได้) แต่เมื่อเราสมมติว่าตัวลายเซ็นถูกทำให้เสียหายหรือถูกดัดแปลง เพียงแค่ 1 Bit (จาก 2F เป็น 3F ใน Byte สุดท้าย) กระบวนการคำนวณ $M' \equiv S^e \pmod{n}$ จะให้ผลลัพธ์ออกมาเป็นค่า Hex ขยะที่ไม่สามารถแปลงกลับเป็นคำว่า "Launch a missle." ได้เลย จากที่กล่าวไปข้างต้น ผู้รับ (Bob) จะทราบทันทีว่าข้อความหรือลายเซ็นมีการถูกดัดแปลงระหว่างทาง กระบวนการ Verify จะล้มเหลว (Fail) ซึ่งเป็นวิธีที่อัลกอริทึม RSA ใช้เพื่อรักษาความน่าเชื่อถือและการป้องกันความสมบูรณ์ของข้อมูล หากแฮกเกอร์ไม่มี Private Key ของ Alice พวกเขาก็ไม่สามารถสร้างลายเซ็นปลอมที่รันผ่านสมการนี้ได้สำเร็จครับ